STAT 153 & 248 - Time Series
Lecture Twenty One

Fall 2026, UC Berkeley

Aditya Guntuboyina

10 November 2026

1 ARMA models

The ARMA(p, q) model is defined by the equation:

$$(y_t - \mu) - \phi_1 (y_{t-1} - \mu) - \cdots - \phi_p (y_{t-p} - \mu) = \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q} \tag{1}$$

where, as usual, $\epsilon_t \overset{\text{i.i.d}}{\sim} N(0, \sigma^2)$. The unknown parameters in this model are $\mu, \phi_1, \ldots, \phi_p, \theta_1, \ldots, \theta_q$ and the noise standard deviation σ.

An equivalent way of writing (1) is by moving the μ terms to the right hand side. This gives:

$$y_t - \phi_1 y_{t-1} - \cdots - \phi_p y_{t-p} = \mu (1 - \phi_1 - \cdots - \phi_p) + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}. \tag{2}$$

In other words,

$$y_t = \phi_0 + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \eta_t \tag{3}$$

where $\phi_0 = \mu(1 - \phi_1 - \cdots - \phi_p)$ and $\eta_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$. In this sense, ARMA($p$, q) can be seen as an AR(p) model where the error terms are modeled as MA(q) instead of i.i.d Gaussian $N(0, \sigma^2)$.

AR(p) and MA(q) models are special cases of ARMA(p, q) because:

1. When $p = 0$, the equation (1) becomes:

   $$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$$

   which is the MA(q) model. So ARMA(0, q) = MA(q).

2. When $q = 0$, the equation (2) becomes:

   $$y_t = \phi_0 + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \epsilon_t$$

   which is the AR(p) equation with $\phi_0 = \mu(1 - \phi_1 - \cdots - \phi_p)$. Thus ARMA($p$, 0) = AR($p$).

2 Backshift Notation

A convenient piece of notation used while working with AR and MA models is the Backshift notation. Let B denote the *backshift operator* defined by

$$By_t = y_{t-1}, B^2y_t = y_{t-2}, B^3y_t = y_{t-3}, \ldots$$

and similarly

$$B\epsilon_t = \epsilon_{t-1}, B^2\epsilon_t = \epsilon_{t-2}, B^3\epsilon_t = \epsilon_{t-3}, \ldots.$$

Also let I denote the identity operator: $Iy_t = y_t$. More generally, we can define polynomial functions of the Backshift operator by, for example,

$$(I + B + 3B^2)y_t = Iy_t + By_t + 3B^2y_t = y_t + y_{t-1} + 3y_{t-2}.$$

In general, for every polynomial $f(z)$, we can define $f(B)$. One can even extend this notation to negative powers of B which correspond to forward shifts. For example, $B^{-1}y_t = y_{t+1}, B^{-5}y_t = y_{t+5}$ and $(B^3 + 9B^{-2})y_t = y_{t-3} + 9y_{t+2}$ etc.

In this notation, the defining equation $y_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \epsilon_t$ for the $AR(p)$ model can be written as $\phi(B)y_t = \phi_0 + \epsilon_t$ for the polynomial $\phi(z) = 1 - \phi_1 z - \phi_2 z^2 - \cdots - \phi_p z^p$.

The defining equation $y_t = \epsilon_t + \theta_1\epsilon_{t-1} + \cdots + \theta_q\epsilon_{t-q}$ for the MA(q) model becomes $y_t = \theta(B)\epsilon_t$ for the polynomial $\theta(z) = 1 + \theta_1 z + \ldots \theta_q z^q$.

The defining equation (1) for the ARMA(p, q) model (1) becomes

$$\phi(B)(y_t - \mu) = \theta(B)\epsilon_t$$

The operator $I - B$ is known as the differencing operator and has the special notation $\nabla = I - B$. Observe that

$$\nabla y_t = (I - B)y_t = y_t - y_{t-1}.$$

In other words, ∇ applied to a time series y_t produces differences $y_t - y_{t-1}$. This is the reason why ∇ is called the differencing operator. Applying ∇ twice produces second order differences:

$$\begin{aligned}\nabla^2 y_t &= \nabla(\nabla y_t) \\ &= \nabla(y_t - y_{t-1}) = \nabla y_t - \nabla y_{t-1} = (y_t - y_{t-1}) - (y_{t-1} - y_{t-2}) = y_t - 2y_{t-1} + y_{t-2}.\end{aligned}$$

Higher order differences $\nabla^k y_t$ are defined recursively. Note that the length of the time series comes down after each successive differencing. For example, ∇y_t has length $n - 1$, $\nabla^2 y_t$ has length $n - 2$ and so on.

3 ARIMA models

ARIMA stands for AutoRegressive Integrated Moving Average. ARIMA is essentially differencing plus ARMA.

Definition 3.1 (ARIMA)**.** *A time series model y_t is said to be ARIMA(p, d, q) if*

$$\phi(B)((\nabla^d y_t) - \mu) = \theta(B)\epsilon_t,$$

where $\epsilon_t \overset{i.i.d}{\sim} N(0, \sigma^2)$.

ARIMA models are fit by the function `ARIMA()` in `statsmodels`. The mean μ above is taken to be zero by default when the order parameter d in `ARIMA` is strictly larger than zero.

When does it make sense to difference the data? To understand this, consider the AR(1) model $y_t - \mu = \phi_1(y_{t-1} - \mu) + \epsilon_t$. When $|\phi_1| < 1$, only a fraction of the deviation of y_{t-1} from μ is carried over to time t. So whenever the series moves away from μ, it gets pulled back towards μ, and the effect of any shock ϵ_t fades away with time. This is a good model for a series which fluctuates around a fixed level. But many real-world time series (e.g., stock prices, GDP, population, price levels) do not behave like this: they have no fixed level to which they keep returning, and instead wander away for long periods of time, often with a trend. When $|\phi_1| > 1$, we have the opposite problem: deviations from μ get magnified at every time step, and the series explodes exponentially. Such behavior is also rarely seen in real data. The boundary case $\phi_1 = 1$ gives

$$y_t = y_{t-1} + \epsilon_t,$$

which is the **random walk** model (note that μ drops out). Here the deviations are carried over fully, neither shrinking nor growing. We can also write this model as

$$y_t = y_0 + \epsilon_1 + \epsilon_2 + \cdots + \epsilon_t,$$

which shows that every shock permanently shifts the level of the series. There is nothing pulling the series back to any particular level, and it is free to wander. The best prediction for future values is simply the current value y_t, as opposed to the case $|\phi_1| < 1$, where predictions are pulled towards μ.

The random walk model is the same as saying that the differenced series $\nabla y_t = y_t - y_{t-1}$ equals ϵ_t. The ARIMA(p, 1, q) model generalizes this by allowing the changes ∇y_t to follow an ARMA(p, q) model instead of being i.i.d. So differencing is technically a special case of the AR model where a coefficient is fixed at 1 instead of being estimated. The series y_t is then obtained from the changes by summing them: $y_t = y_0 + \nabla y_1 + \cdots + \nabla y_t$. This is the reason for the word "integrated" in ARIMA.

In practice, differencing ($d = 1$) makes sense when the series does not seem to fluctuate around a fixed level (it shows trends or long excursions, and its typical level in one stretch of time is quite different from that in another stretch), while the changes $y_t - y_{t-1}$ do seem to fluctuate around a fixed level. It is also often the case that the changes are the more meaningful quantity: for example, when y_t is the logarithm of a stock price, the changes are the returns, and when y_t is the logarithm of GDP, the changes are the growth rates. On the other hand, if the series itself keeps returning to a fixed level, then one should use $d = 0$.

When $d = 1$, the ARIMA(p, 1, q) model becomes:

$$\phi(B)\,(\nabla y_t - \mu) = \theta(B)\epsilon_t. \tag{4}$$

Define a process η_t by $y_t = y_0 + \mu t + \eta_t$ (in other words, $\eta_t = y_t - y_0 - \mu t$). Then

$$\nabla y_t = y_t - y_{t-1} = (y_0 + \mu t + \eta_t) - (y_0 + \mu(t-1) + \eta_{t-1}) = \mu + (\eta_t - \eta_{t-1}) = \mu + \nabla\eta_t.$$

Plugging this into (4), we get

$$\phi(B)\nabla\eta_t = \theta(B)\epsilon_t.$$

In other words, (4) is equivalent to:

$$y_t = y_0 + \mu t + \eta_t \qquad \text{where } (\nabla\eta_t) \text{ is ARMA}(p, q) \text{ with zero mean.}$$

When $\mu \neq 0$, the term $y_0 + \mu t$ represents a deterministic linear trend in y_t. However many real-world time series are unlikely to exhibit an exact deterministic linear trend. For this reason, the default behavior of the `ARIMA` function in `statsmodels` sets $\mu = 0$ in (4). To fit it with $\mu \neq 0$, one must specify the argument `trend = 't'`. Note that, even when $\mu \neq 0$, the series y_t does not stay close to the line $y_0 + \mu t$. This is because η_t is itself a wandering series (its differences follow an ARMA model), so y_t can drift arbitrarily far from the line. The parameter μ should therefore be interpreted as the average change per unit time, and not as the slope of a line around which the data fluctuate.

Sometimes differencing once is not enough, because the changes $y_t - y_{t-1}$ themselves wander. This happens when the series is smooth with a slope that changes slowly over time (e.g., a price level whose rate of inflation goes through long periods of being high and long periods of being low). In this case, we use $d = 2$ and model

$$\nabla^2 y_t = (y_t - y_{t-1}) - (y_{t-1} - y_{t-2}),$$

which is the change in the change. The ARIMA(p, 2, q) model says that this quantity fluctuates around zero, i.e.,

$$y_t \approx y_{t-1} + (y_{t-1} - y_{t-2}).$$

In words: to predict the next value, start with the current value and add the most recently observed change. Compare this to $d = 1$ (with $\mu = 0$), where the simplest prediction is the current value itself. Values of d larger than 2 are almost never used.

4 Parameter estimation, AIC and BIC

For parameter estimation in ARIMA(p, d, q) models, we use the `ARIMA` function from `statsmodels`, specifying `order=(p,d,q)` and then calling `fit()`: `mdfit = ARIMA(y, order=(p, d, q)).fit()`. Here `y` is the original series; the function accounts for the d differences internally. Setting $d = 0$ gives an ARMA(p, q) model.

By default, the fitting procedure estimates the model parameters by maximizing the Gaussian likelihood. Evaluating this likelihood is more involved than for AR models, and the function uses ideas from state space modeling and the Kalman filter to carry out this calculation. The likelihood maximization problem is itself nontrivial. In general, the likelihood depends nonlinearly on the model parameters, and there is no closed-form expression for its maximizer. Numerical optimization algorithms are therefore used to search iteratively for parameter values that increase the likelihood. The default algorithm is limited-memory BFGS (`lbfgs`). Other available algorithms include BFGS (`bfgs`), Powell's method (`powell`), Nelder–Mead (`nm`), Newton–Raphson (`newton`), conjugate gradient (`cg`), Newton conjugate gradient (`ncg`), and basin-hopping (`basinhopping`). For example, Powell's method can be selected using:

```
mdfit = ARIMA(y, order=(p, d, q)).fit(method_kwargs={'method': 'powell', 'maxiter': 1000})
```

These algorithms use different strategies to search for the maximum and can differ in speed and convergence behavior. Consequently, trying another optimizer or different starting values can sometimes help when the default fitting procedure encounters numerical difficulties. We will not go into the details of the Kalman filter or these optimization algorithms.

The treatment of constant and trend terms depends on d. By default, a constant term is included when $d = 0$, while no deterministic trend is included when $d > 0$. For $d = 1$,

specifying `trend='t'` includes a linear trend in the original series, corresponding to a nonzero mean for its first differences (often called a drift).

Sometimes the fitting procedure produces warnings or errors. In that case, change either the starting parameters or the optimization algorithm or the number of iterations and try fitting the model again.

For model comparison, AIC and BIC are commonly used. The AIC and BIC for a fitted ARIMA(p, d, q) model are:

$$AIC = -2\widehat{\ell} + 2k, \qquad BIC = -2\widehat{\ell} + (\log n_{\text{eff}})k,$$

where $\widehat{\ell}$ is the maximized log-likelihood, k is the number of estimated parameters, and n_{eff} is the effective number of observations used in the likelihood.

The parameter count includes the p AR coefficients, the q MA coefficients, the shock variance, and any estimated constant or trend coefficients. Thus, without a constant or trend, $k = p + q + 1$; with one additional constant or drift parameter, $k = p + q + 2$. The differencing order d is not counted as an estimated coefficient. In practice, we use `mdfit.aic` and `mdfit.bic`, which apply the appropriate parameter count and effective sample size automatically.

Models with smaller values of AIC and BIC are preferred. The first term measures the quality of fit, while the second penalizes model complexity, helping to guard against overfitting. Generally, $\log n_{\text{eff}}$ is larger than 2, so BIC imposes a stronger penalty on additional parameters and tends to favor simpler models than AIC.

For ARIMA models, we generally choose d first and then use AIC or BIC to compare different values of p and q, using the same observations and response scale. Direct comparisons across different values of d require care because differencing and initialization can change the likelihood being compared. Forecast performance on held-out observations provides another way to compare models with different differencing orders.

5 Seasonal ARMA Models

Seasonal ARMA models are often useful while modeling datasets having seasonal features (e.g., monthly datasets). We say that $\{y_t\}$ is a seasonal ARMA(P, Q) process with period s if it satisfies the difference equation $\Phi(B^s)(y_t - \mu) = \Theta(B^s)\epsilon_t$ where $\epsilon_t \overset{\text{i.i.d}}{\sim} N(0, \sigma^2)$ and

$$\Phi(B^s) = 1 - \Phi_1 B^s - \Phi_2 B^{2s} - \cdots - \Phi_P B^{Ps}$$

and

$$\Theta(B^s) = 1 + \Theta_1 B^s + \Theta_2 B^{2s} + \cdots + \Theta_Q B^{Qs}.$$

The seasonal ARMA(P, Q) model with period s is a special case of an ARMA(Ps, Qs) model. However the seasonal model has $P + Q + 1$ (the 1 is for σ^2) parameters while a general ARMA(Ps, Qs) model will have $Ps + Qs + 1$ parameters. So the seasonal models are much sparser.

Why are these models sensible for seasonal data? Take monthly data ($s = 12$) and consider the simplest case $P = 1, Q = 0$:

$$y_t - \mu = \Phi_1(y_{t-12} - \mu) + \epsilon_t.$$

This is the seasonal AR(1) model which says that the best guide to this January is last January, and not the months immediately preceding it. If last December's sales were unusually high (relative to the overall mean μ), then we expect this December's sales to also be high, with Φ_1 controlling how much of last year's deviation carries over to this year. This is exactly the kind of dependence we expect in seasonal data: observations that are s time points apart are in the same phase of the seasonal cycle and so should resemble each other.

A useful way to think about the general seasonal ARMA(P, Q) model is to split the data into s subseries, one for each season: the series of all Januaries, the series of all Februaries, and so on. Because the difference equation only links y_t with $y_{t-s}, y_{t-2s}, \ldots$ and ϵ_t with $\epsilon_{t-s}, \epsilon_{t-2s}, \ldots$, each of these subseries is just an ordinary ARMA(P, Q) process observed once a year, and all s subseries share the same parameters. The seasonal model is therefore nothing but the usual ARMA model applied year-to-year instead of month-to-month. The AR part says that the value for a given month depends on the values for the same month in the previous P years. The MA part says that a shock in a given month (e.g., an unusually harsh winter month) continues to affect the same month for the next Q years and then disappears.

This model however ignores short-term dependence altogether: because the s subseries are driven by disjoint sets of the ϵ_t's, they are independent of each other, so that, for example, January and February of the same year are independent. This is unrealistic for most datasets, and is the reason why seasonal ARMA models are, in practice, combined with ordinary ARMA models (which capture the month-to-month dependence).

6 Multiplicative Seasonal ARMA models

The **Multiplicative Seasonal Autoregressive Moving Average Model** ARMA(p, q) $\times$ $(P, Q)_s$ is defined via the difference equation:

$$\Phi(B^s)\phi(B)(y_t - \mu) = \Theta(B^s)\theta(B)\epsilon_t.$$

Here $\phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ and $\theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$ are the usual AR and MA polynomials, and $\Phi(B^s)$ and $\Theta(B^s)$ are the seasonal polynomials from the previous section. When $P = Q = 0$, this reduces to the usual ARMA(p, q) model, and when $p = q = 0$, it reduces to the seasonal ARMA(P, Q) model.

Why multiply the polynomials? The most natural way to see this is to build the model in two stages. Recall that the problem with the pure seasonal model

$$\Phi(B^s)(y_t - \mu) = \Theta(B^s)u_t$$

is that the driving noise u_t is assumed to be i.i.d, which makes different months of the same year independent. The seasonal model does a good job of explaining the year-to-year dependence; what is left over after this is explained (namely u_t) is what we assumed to be white noise. The obvious fix is to drop this assumption and allow the leftover u_t to have month-to-month dependence, which we know how to model: with an ordinary ARMA(p, q) model

$$\phi(B)u_t = \theta(B)\epsilon_t,$$

where now ϵ_t is i.i.d. Applying $\phi(B)$ to both sides of the seasonal equation and using the ARMA equation for u_t gives

$$\phi(B)\Phi(B^s)(y_t - \mu) = \Theta(B^s)\phi(B)u_t = \Theta(B^s)\theta(B)\epsilon_t,$$

which is exactly the multiplicative model. So the product of polynomials arises simply because we are applying two models one after the other: a seasonal ARMA model whose errors follow a regular ARMA model. Since the polynomials in B commute, we could equally well have done this in the opposite order: start with a regular ARMA model for the month-to-month dependence, and let its errors follow a seasonal ARMA model to capture the leftover year-to-year dependence. Both descriptions lead to the same model, so neither kind of dependence is given priority over the other.

To see concretely what the multiplicative structure does, consider the ARMA(1, 0) $\times$ $(1, 0)_{12}$ model for monthly data: $(1 - \Phi B^{12})(1 - \phi B)(y_t - \mu) = \epsilon_t$. Writing $\tilde{y}_t = y_t - \mu$ and expanding the product, we get

$$\tilde{y}_t = \phi\tilde{y}_{t-1} + \Phi\tilde{y}_{t-12} - \phi\Phi\tilde{y}_{t-13} + \epsilon_t.$$

The terms $\phi\tilde{y}_{t-1}$ and $\Phi\tilde{y}_{t-12}$ are expected: this month depends on last month and on the same month last year. The cross term involving $\tilde{y}_{t-13}$ may seem strange at first, but it has a simple explanation. Rearranging the equation gives

$$\tilde{y}_t - \Phi\tilde{y}_{t-12} = \phi\left(\tilde{y}_{t-1} - \Phi\tilde{y}_{t-13}\right) + \epsilon_t.$$

The quantity $\tilde{y}_t - \Phi\tilde{y}_{t-12}$ is the part of this month's value that is *not* explained by the same month last year, i.e., it is how surprising this month is given last year. The equation says that these surprises follow an AR(1) model: if last month was higher than what was expected based on the previous year, then this month will tend to be higher than expected as well. For example, if this November's sales are well above what last November would suggest, then we expect this December's sales to also be above what last December would suggest.

The same phenomenon occurs on the moving average side. In the ARMA(0, 1) $\times$ $(0, 1)_{12}$ model, we have

$$y_t - \mu = (1 + \Theta B^{12})(1 + \theta B)\epsilon_t = \epsilon_t + \theta\epsilon_{t-1} + \Theta\epsilon_{t-12} + \theta\Theta\epsilon_{t-13}.$$

A shock has two channels through which it affects the future: it carries over to the next month with weight θ, and to the same month next year with weight Θ. The cross term $\theta\Theta\epsilon_{t-13}$ corresponds to the shock passing through both channels. If a shock in a given month shows up again one year later with weight Θ, and if anything that happens in one month spills over into the next month with weight θ, then the shock should also show up thirteen months later with weight $\theta\Theta$. It would be odd for it not to.

The multiplicative ARMA(p, q) $\times$ $(P, Q)_s$ model is a special case of the ARMA($p + Ps$, $q+Qs$) model. But it has only $p+q+P+Q+1$ parameters, as opposed to $p+q+Ps+Qs+1$ for the general model. The coefficients of the cross terms (such as $-\phi\Phi$ and $\theta\Theta$ above) are not free parameters; they are determined by the regular and seasonal coefficients. So the multiplicative structure gives us both kinds of dependence, along with sensible interactions between them, without any additional parameters.

7 SARIMA Models

These models are obtained by combining differencing with multiplicative seasonal ARMA models. These models are denoted by ARIMA(p, d, q) $\times$ $(P, D, Q)_s$. This means that after differencing d times and seasonal differencing D times (with period s), we get a multiplicative seasonal ARMA model. In other words, $\{y_t\}$ is ARIMA(p, d, q) $\times$ $(P, D, Q)_s$ if it satisfies the difference equation:

$$\Phi(B^s)\phi(B)\nabla_s^D\nabla^d(y_t - \mu) = \Theta(B^s)\theta(B)\epsilon_t. \tag{5}$$

Recall that $\nabla_s^D = (1 - B^s)^D$ and $\nabla^d = (1 - B)^d$ denote the differencing operators.

In practice, d and D are almost always taken to be 0 or 1, and the most common choice for seasonal data with a trend is $d = D = 1$. To understand when this choice makes sense, let us look at what each kind of differencing does (take $s = 12$ for concreteness).

As discussed in the previous section, regular differencing ($d = 1$) is appropriate when the level of the series wanders over time instead of fluctuating around a fixed level.

Seasonal differencing ($D = 1$) is appropriate when the seasonal pattern is strong and persistent. The reasoning is the same as that for regular differencing, applied year-to-year instead of month-to-month. In the seasonal AR(1) model $y_t - \mu = \Phi_1(y_{t-12} - \mu) + \epsilon_t$ with $|\Phi_1| < 1$, only a fraction of the deviation of a given month from μ is carried over to the same month of the next year. So seasonal deviations fade away over the years, and every month keeps getting pulled back to the same level μ. This is not suitable for a dataset where, say, every December is much higher than every July, year after year. When $|\Phi_1| > 1$, the seasonal deviations explode, which is also unrealistic. For a persistent seasonal pattern, we want the seasonal deviations to be carried over fully, which corresponds to the boundary case $\Phi_1 = 1$. The model then becomes $y_t - y_{t-12} = \epsilon_t$, i.e., it is a model for the seasonal difference $\nabla_{12} y_t = y_t - y_{t-12}$. Just as in the case of regular differencing, $D = 1$ corresponds to fixing this coefficient at 1 instead of estimating it. The seasonal difference compares each month with the same month of the previous year, and this removes any persistent differences between the months. This is the familiar "year-over-year change" that is commonly used for reporting economic data.

Many datasets have both a trend and a strong seasonal pattern. Taking only the regular difference will not suffice for such data, because the month-to-month changes $y_t - y_{t-1}$ will still be seasonal (e.g., the change from November to December is large and positive every year). Taking only the seasonal difference removes the seasonal pattern, but the year-over-year change $y_t - y_{t-12}$ is essentially the growth of the series over the past year. If this growth fluctuates around a fixed value, then $D = 1$ alone is enough. But often the growth itself wanders over time (periods of fast growth followed by periods of slow growth), and then $y_t - y_{t-12}$ needs to be differenced again. This leads to $d = D = 1$:

$$\nabla\nabla_{12} y_t = (y_t - y_{t-12}) - (y_{t-1} - y_{t-13}),$$

which is the change in the year-over-year change from last month to this month. The model with $d = D = 1$ says that this quantity fluctuates around zero without any trend or persistent seasonal pattern, i.e.,

$$y_t \approx y_{t-12} + (y_{t-1} - y_{t-13}).$$

In words: to predict this month, start with the same month last year and add the most recently observed year-over-year growth. This is how one would intuitively forecast a series with trend and seasonality: y_{t-12} takes care of the seasonal pattern, and $y_{t-1} - y_{t-13}$ takes care of how much the series has grown over the past year. The multiplicative seasonal ARMA part of the model then describes the errors in this simple prediction rule. A famous example is the ARIMA(0, 1, 1) $\times$ $(0, 1, 1)_{12}$ model which was used by Box and Jenkins for the (logarithms of the) monthly airline passenger data.

8 Fitting SARIMA models

The model (5) can be fit to the data using the function `ARIMA` in `statsmodels` with the `seasonal_order` argument: `ARIMA(y, order = (p, d, q), seasonal_order = (P, D, Q,`

`s))`. To use it, we need to choose the orders p, d, q, P, D, Q (the period s is known from the context, e.g., $s = 12$ for monthly data). We shall first fix d and D based on the considerations described above (e.g., $d = D = 1$). With d and D fixed, we then cycle over a grid of small values of p, q, P, Q (e.g., $p, q \in \{0, 1, 2, 3\}$ and $P, Q \in \{0, 1\}$), fit the model for each combination, and pick the combination with the smallest value of AIC or BIC. Both criteria reward models which fit the data well and penalize models with a large number of parameters, with BIC penalizing more heavily and hence tending to pick smaller models. The reason for not including d and D in this search is that AIC and BIC can only be used to compare models which are fit to the same data: models with different values of d or D are effectively fit to different differenced series, and so their AIC and BIC values are not directly comparable.